

RISING WATERS

ML-Based Flood Risk Prediction Web Application

Project Report

Submitted by

Sravani

Computer Science & Engineering

Rajiv Gandhi University of Knowledge Technologies (RGUKT), RK Valley

AI and ML Internship — SmartBridge

TABLE OF CONTENTS

1. Introduction.....	4
1.1 Project Overview.....	4
1.2 Purpose.....	4
2. Ideation Phase.....	5
2.1 Problem Statement.....	5
2.2 Empathy Map Canvas.....	5
2.3 Brainstorming.....	5
3. Requirement Analysis.....	6
3.1 Customer Journey Map.....	6
3.2 Solution Requirement.....	6
3.3 Data Flow Diagram.....	6
3.4 Technology Stack.....	6
4. Project Design.....	8
4.1 Problem Solution Fit.....	8
4.2 Proposed Solution.....	8
4.3 Solution Architecture.....	8
5. Project Planning & Scheduling.....	9
5.1 Project Planning.....	9
6. Functional and Performance Testing.....	10
6.1 Performance Testing.....	10
7. Results.....	11
7.1 Output Screenshots.....	11
8. Advantages & Disadvantages.....	13
9. Conclusion.....	14
10. Future Scope.....	14
11. Appendix.....	15

1. INTRODUCTION

1.1 Project Overview

Rising Waters is an end-to-end machine learning system that predicts flood risk from everyday weather readings. A user (a resident, a local disaster-management officer, or a station operator) enters five weather parameters — annual rainfall, seasonal rainfall, cloud visibility, temperature, and humidity — into a web form, and the system returns an immediate flood risk / no-flood-risk verdict.

The project covers the complete pipeline expected of a production-style ML application: data collection, exploratory data analysis, preprocessing, comparison of multiple classification algorithms, selection of the best-performing model, and deployment behind a Flask web interface with a clean, station-dashboard-style front end.

1.2 Purpose

Floods are among the most frequent and damaging natural disasters, and early warning is one of the most effective ways to reduce loss of life and property. Official flood forecasting systems exist, but they are often not accessible in a simple, instant form to individuals or small local authorities who only have basic weather readings on hand.

The purpose of this project is to:

- Provide a lightweight, instant flood-risk estimate from a small set of easily obtainable weather inputs.
- Demonstrate a complete, reproducible ML workflow — from raw data to a deployed web application.
- Compare multiple classification algorithms objectively and justify the final model choice with metrics.
- Package the solution so it can run on modest hardware (a single Flask process, no external inference servers).

2. IDEATION PHASE

2.1 Problem Statement

Residents and local authorities in flood-prone regions frequently lack a fast, easy-to-use way to judge whether current weather conditions carry a meaningful flood risk. Existing forecasting dashboards are often built for meteorologists, are region-specific, or require an internet connection to a live data feed. As a result, many people rely on visible signs (already-rising water, heavy ongoing rain) rather than an early, data-driven signal, which shortens the time available to prepare, protect valuables, or evacuate.

"How might we let anyone convert a handful of everyday weather readings into a clear, immediate flood-risk signal — without specialist tools or a live data subscription?"

2.2 Empathy Map Canvas

The target user considered for this canvas is a resident of a semi-urban / rural flood-prone locality who follows local rainfall reports.

Quadrant	Insights
SAYS	"I only find out it's serious once the water is already at my door." "The news warns whole districts, not my street."
THINKS	Wonders if this season's rainfall pattern is unusual. Worries about deciding too late to move belongings or livestock to safety.
DOES	Checks local news and rain-gauge readings informally. Asks neighbours and elders for their read on the weather.
FEELS	Anxious during the monsoon season. Frustrated by generic, region-wide alerts that don't reflect local conditions.
PAIN	No simple tool to combine rainfall, humidity, temperature, and visibility into one risk verdict. Warnings arrive late or are too broad.
GAIN	A fast, on-demand risk check using numbers the user already has (or can look up in a minute). Confidence to act early on packing, moving valuables, or contacting authorities.

2.3 Brainstorming

Ideas generated and grouped during brainstorming, with the selected direction highlighted:

- SMS/IVR-based alert system triggered by a government weather feed — rejected: depends on external live data access and telecom integration, out of scope for the internship timeline.
- Mobile app with GPS-based automatic weather lookup — rejected for this phase: requires a paid weather API and app-store distribution.

- Browser-based form + trained ML classifier that scores manually entered weather readings — selected: no live data dependency, works offline once trained, fastest to prototype, teaches the full ML pipeline end-to-end.
- Rule-based threshold system (e.g. "if rainfall > X then flood") — rejected as primary approach: simplistic, ignores interaction between variables (e.g. high rainfall with high cloud visibility is less risky than the same rainfall with low visibility and high humidity); used only as a sanity check against the trained model.

The selected direction — a Flask web app backed by a trained classifier — was chosen because it best balances feasibility within the internship timeline, educational depth (covers EDA, preprocessing, multi-model comparison, and deployment), and genuine usefulness to a non-technical end user.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

Stage	User Action	Experience
Awareness	Hears about the Rising Waters tool from a local notice board / word of mouth.	Curious whether a simple web tool can really help.
Landing	Opens the home page.	Sees a clear explanation of what the tool does and how it works (rainfall → scaling → prediction).
Input	Clicks "Run a prediction" and fills in five weather values.	Finds the form short and self-explanatory, with units and placeholders shown for every field.
Decision	Submits the form.	Waits under a second for the result — no confusing intermediate screens.
Result	Sees either "Flood risk detected" or "No flood risk detected."	Gets a plain-language verdict plus concrete next-step suggestions (in the flood-risk case).
Follow-up	Cross-checks with official alerts if risk was flagged, or returns later with updated readings.	Feels informed early enough to act, rather than reacting only when water is already visible.

3.2 Solution Requirement

Functional Requirements:

- FR1: The system shall accept five numeric weather inputs — annual rainfall, seasonal rainfall, cloud visibility, temperature, and humidity.

- FR2: The system shall validate that all fields are filled with valid numbers before running a prediction.
- FR3: The system shall scale the input using the same StandardScaler fitted during training.
- FR4: The system shall classify the scaled input using the trained model and return one of two outcomes: flood risk / no flood risk.
- FR5: The system shall present the outcome on a dedicated result page with guidance appropriate to that outcome.

Non-Functional Requirements:

- NFR1 (Performance): A prediction shall be returned in well under one second on standard hardware.
- NFR2 (Usability): The form shall be usable without technical training, with units labelled on every field.
- NFR3 (Portability): The application shall run as a single Flask process with no external API dependency at inference time.
- NFR4 (Maintainability): The model and scaler shall be swappable by re-running the training script, without touching the Flask application code.
- NFR5 (Reliability): Malformed or missing form input shall be handled gracefully with an inline error rather than a server crash.

3.3 Data Flow Diagram

Level-0 data flow: the user submits weather readings through the browser form; the Flask backend loads the persisted StandardScaler and XGBoost model at startup; on each request, the input is scaled, passed to the model, and the resulting class is mapped to a redirect toward either the "Flood risk" or "No flood risk" result page.

User (Browser) → fills form (5 weather fields) → **Flask /predict** → StandardScaler.transform() → XGBoost.predict() → class 0/1 → redirect → **chance.html / no_chance.html**

Supporting offline flow (training pipeline, run separately, not on the request path): flood_dataset.csv → load_data() → EDA plots → preprocess() (median impute, IQR outlier capping, train/test split, feature scaling) → train & compare four classifiers → save best model as floods.save and the fitted scaler as transform.save.

3.4 Technology Stack

Layer	Technology	Purpose
Language	Python 3.8+	Core implementation language for both the ML pipeline and the web backend.
Data handling	Pandas, NumPy	Loading, cleaning, and transforming the tabular weather dataset.
Visualization (EDA)	Matplotlib, Seaborn	Distribution plots, box plots, and correlation heat map.

Layer	Technology	Purpose
Machine learning	Scikit-learn (Decision Tree, Random Forest, KNN, StandardScaler)	Baseline classifiers, preprocessing utilities, evaluation metrics.
Machine learning	XGBoost	Gradient-boosted classifier; selected as the final production model.
Model persistence	Joblib	Serializing the trained model (floods.save) and scaler (transform.save).
Web framework	Flask	Serves the web application, form handling, and routing.
Frontend	HTML5, CSS3, vanilla JavaScript	Home, prediction-form, and result pages (chance.html / no_chance.html).
Environment	Ubuntu Linux, virtualenv	Reproducible runtime with pinned requirements.txt.

4. PROJECT DESIGN

4.1 Problem – Solution Fit

The core problem is a gap between raw, locally available weather readings and an actionable flood-risk decision. Rather than requiring a live data feed or specialist interpretation, Rising Waters closes this gap with a trained classifier that has learned the (non-linear, interacting) relationship between rainfall, visibility, temperature, humidity, and flood outcomes from historical-style data. The fit is validated by the model's ability to separate flood from non-flood cases with 85% test accuracy — well above a naive baseline — while requiring only five simple inputs a user can obtain from a local weather report.

4.2 Proposed Solution

Aspect	Description
Problem Statement	Give ordinary users an instant, data-driven flood-risk verdict from a handful of weather readings.
Idea / Solution	A Flask web app front-ending a trained XGBoost classifier, wrapped in a simple 3-step flow: enter readings → scale & predict → view verdict.
Novelty / Uniqueness	Combines the full ML lifecycle (EDA → preprocessing → multi-model comparison) with a lightweight, dependency-free deployment that runs entirely offline at inference time.
Social Impact / Value	Earlier, self-serve awareness of flood risk for individuals and local communities, encouraging proactive preparation rather than reactive response.
Business / Scalability Model	Deployable as a free community tool by local disaster-management bodies; extensible to SMS/IVR delivery or a public API in future phases.
Scalability	Stateless Flask app behind a WSGI server; the model/scaler pair can be retrained on richer, region-specific data without code changes.

4.3 Solution Architecture

The application follows a simple three-tier architecture:

- Presentation tier: Jinja2-rendered HTML pages (home.html, index.html, chance.html, no_chance.html) styled with main.css and enhanced with main.js.
- Application tier: Flask routes in app.py handle GET /, GET /Predict, POST /predict, and the two result routes; input validation and orchestration of scaling + inference happen here.

- Model tier: A persisted StandardScaler (transform.save) and XGBoost classifier (floods.save), both produced offline by train_model.py and loaded once at Flask startup for fast, stateless inference.

Request flow: Browser → Flask route (/predict) → StandardScaler.transform(input) → XGBoost.predict(scaled_input) → redirect to /chance or /no_chance → rendered result page.

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

The project was organised into four epics, tracked as sprints over the internship duration:

Sprint / Epic	User Stories	Key Deliverables
Epic 1 — Data Collection	As a developer, I need a representative weather-to-flood dataset so the model has something meaningful to learn from.	flood_dataset.csv, generate_dataset.py
Epic 2 — Visualizing & Analysing the Data	As a developer, I need to understand feature distributions, outliers, and correlations before modelling.	eda_distribution.png, eda_boxplots.png, eda_heatmap.png
Epic 3 — Data Pre-processing	As a developer, I need clean, scaled, split data so every model is trained and compared fairly.	Missing-value imputation, IQR outlier capping, train/test split, StandardScaler
Epic 4 — Model Building	As a developer, I need to compare multiple algorithms and keep the best one so predictions are as accurate as possible.	model_comparison.txt, floods.save, transform.save
Epic 5 — Application Building	As a user, I want a simple web form and clear result page so I can check flood risk without technical knowledge.	app.py, home.html, index.html, chance.html, no_chance.html
Epic 6 — Testing & Documentation	As a developer, I need to verify functional correctness and document setup so the project is reproducible.	Functional test pass, README.md, requirements.txt

Each epic was completed sequentially, since later epics (model building, application building) depend directly on the artefacts produced by earlier ones (cleaned data, fitted scaler).

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

Model performance — four classifiers were trained on an 80/20 stratified split of the (cleaned, scaled) dataset and evaluated on the held-out 300-row test set:

Model	Accuracy	Precision (class 1)	Recall (class 1)	F1-score (class 1)
Decision Tree	80.33%	0.73	0.81	0.77
Random Forest	84.00%	0.81	0.78	0.80
K-Nearest Neighbors	84.00%	0.80	0.80	0.80
XGBoost (selected)	85.00%	0.81	0.82	0.81

XGBoost achieved the highest overall accuracy (85.00%) and the best balance of precision and recall on the flood class, and was therefore selected as the production model saved to `floods.save`.

Application / functional testing:

Test Case	Steps	Expected Result	Status
Home page loads	Navigate to /	Landing page renders with project overview and "Run a prediction" call-to-action.	Pass
Prediction form loads	Navigate to /Predict	Form renders with all five labelled input fields.	Pass
Valid input → flood risk	Submit values corresponding to a high-rainfall / high-risk scenario	Redirects to /chance with the flood-risk message and next-step guidance.	Pass
Valid input → no risk	Submit values corresponding to a low-rainfall / low-risk scenario	Redirects to /no_chance with the reassuring message.	Pass
Missing field	Submit the form with one field left blank	Form re-renders with an inline validation error; no server crash.	Pass
Non-numeric input	Enter text in a numeric field	Form re-renders with an inline validation error.	Pass
Model/scaler load	Start the Flask app	<code>floods.save</code> and <code>transform.save</code> load without error at startup.	Pass

7. RESULTS

7.1 Output Screenshots

Exploratory Data Analysis

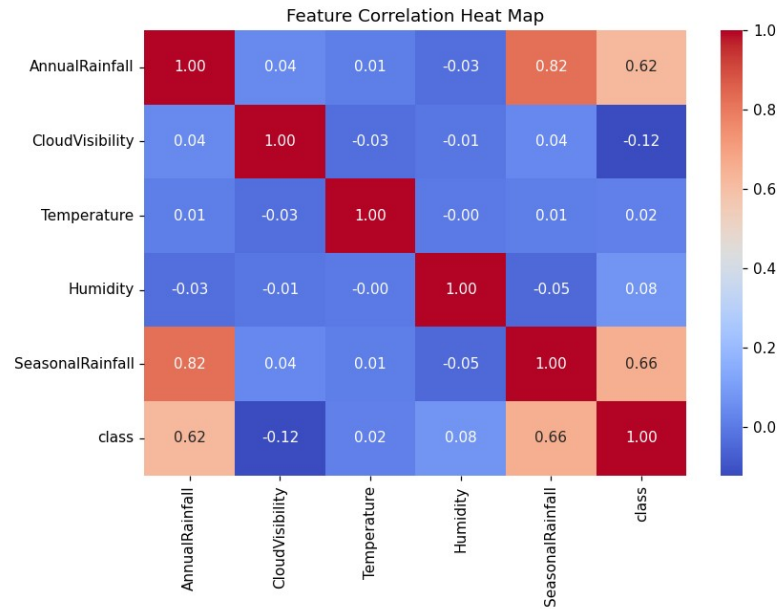


Fig. 7.1 — Feature correlation heat map. Annual and seasonal rainfall show the strongest positive correlation with the flood class (0.62 and 0.66 respectively).

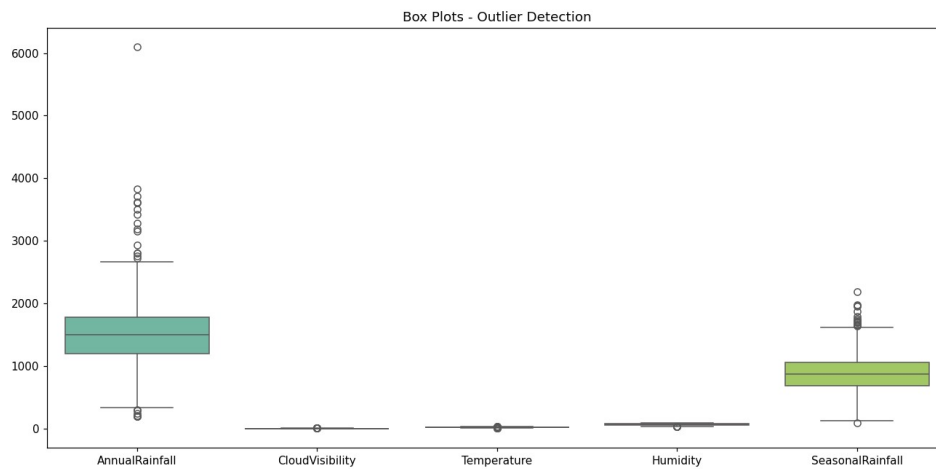


Fig. 7.2 — Box plots used for outlier detection across all five weather features prior to IQR capping.

Web Application

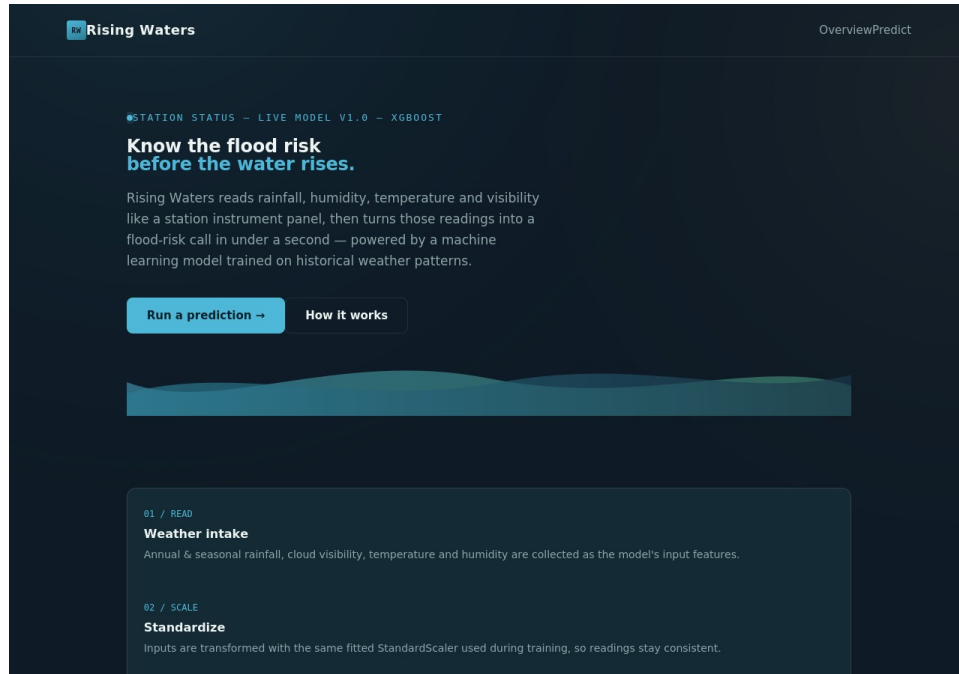


Fig. 7.3 — Home page: project overview and entry point to the prediction form.

 The screenshot shows the prediction form of the 'Rising Waters' web application. The header is the same as the home page. The main content area has a dark blue background. It features a heading 'STATION READOUT' and 'FEATURES: 5 - MODEL: XGB00ST'. Below this is a heading 'Enter today's weather parameters'. There are five input fields, each with a label and a placeholder value: 'ANNUAL RAINFALL (mm)' with 'e.g. 1500', 'SEASONAL RAINFALL (mm, Jun-Sep)' with 'e.g. 900', 'CLOUD VISIBILITY (km)' with 'e.g. 5.5', 'TEMPERATURE (°C)' with 'e.g. 28', and 'HUMIDITY (%)' with 'e.g. 72'. Each input field has a dropdown arrow on the right. At the bottom, there is a 'Run prediction ->' button and a note 'VALUES ARE SCALED WITH THE SAVED STANDARDSCALER BEFORE INFERENCE'.

Fig. 7.4 — Prediction form: the five weather-parameter inputs (annual rainfall, seasonal rainfall, cloud visibility, temperature, humidity).

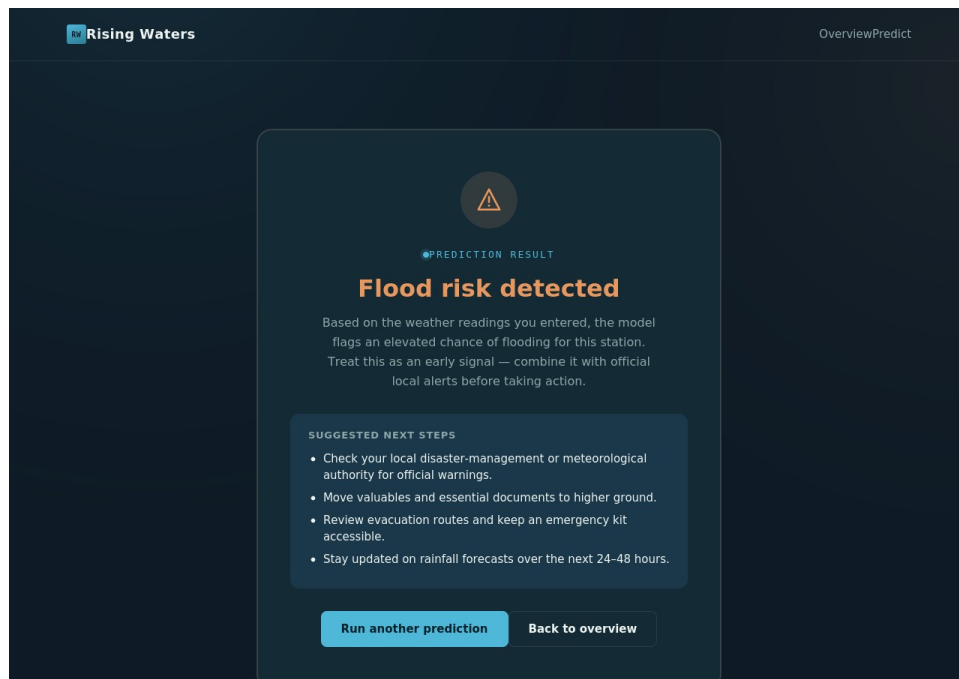


Fig. 7.5 — Result page shown when the model flags flood risk, with suggested next steps.

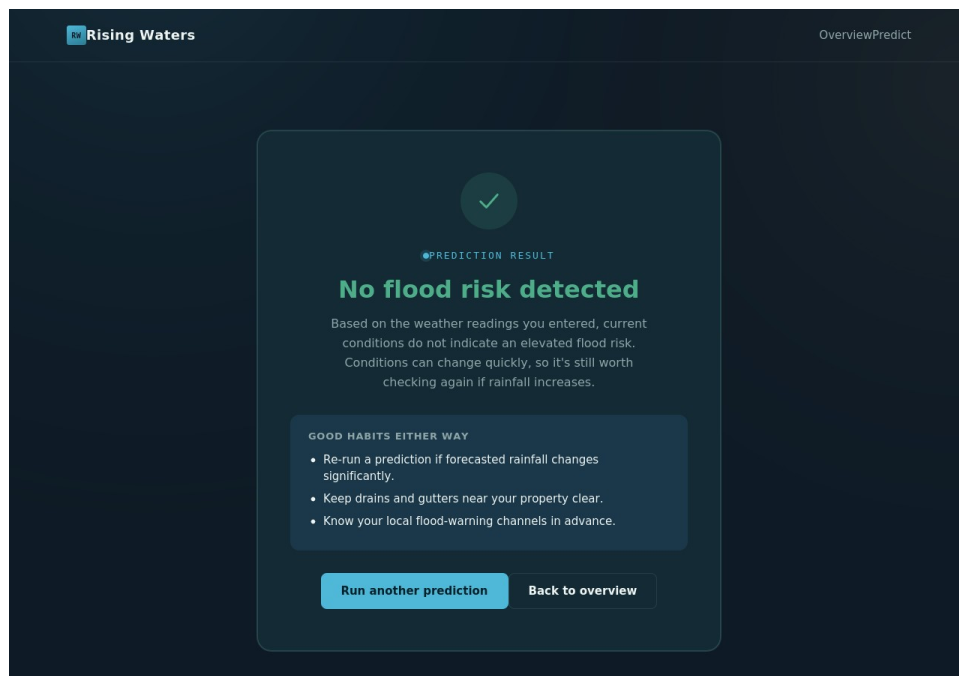


Fig. 7.6 — Result page shown when the model detects no meaningful flood risk.

8. ADVANTAGES & DISADVANTAGES

Advantages

- Instant, self-serve flood-risk verdict from only five easily obtainable weather readings.
- No live external data dependency at inference time — the trained model and scaler are bundled with the app.
- Simple, well-documented pipeline (EDA → preprocessing → multi-model comparison) makes retraining on new or region-specific data straightforward.
- Model choice is backed by a transparent, four-way accuracy comparison rather than a single unverified classifier.
- Lightweight Flask deployment runs on modest hardware and needs no GPU.

Disadvantages

- Trained on a synthetic, statistically-generated dataset (the referenced Kaggle dataset requires authenticated download); real-world accuracy will depend on retraining with authentic regional data.
- Model accuracy (85%) leaves a meaningful margin of error and should not be the sole basis for evacuation decisions.
- Does not account for real-time factors such as river-gauge levels, drainage capacity, or terrain, since the model only uses five weather variables.
- Manual data entry introduces the possibility of user input error, with no automatic cross-check against live weather feeds.

9. CONCLUSION

Rising Waters demonstrates a complete, reproducible machine-learning workflow — from data preparation and exploratory analysis through multi-model comparison to a deployed, user-facing web application — applied to the practical problem of early flood-risk awareness. Among the four classifiers evaluated, XGBoost delivered the strongest and most balanced performance (85% accuracy, 0.81 F1-score on the flood class) and was selected as the production model. The resulting Flask application turns that model into an accessible tool: a short form, an instant verdict, and clear next-step guidance, with no specialist knowledge or live data connection required from the end user. The project met its internship objectives of covering the full ML lifecycle while producing a genuinely usable, deployable artifact.

10. FUTURE SCOPE

- Retrain on authentic, region-specific historical rainfall and flood-incident data (e.g. the referenced Kaggle rainfall dataset) to improve real-world accuracy.
- Integrate a live weather-API feed so users can get a prediction without manually looking up current readings.
- Add SMS/IVR-based alerting for users without regular internet access, building on the brainstorming ideas discarded in the ideation phase.
- Incorporate additional signals such as river-gauge levels, elevation/terrain data, and drainage infrastructure quality for a richer, more localised risk model.
- Expose the prediction endpoint as a documented public API so other local-government tools can integrate flood-risk scoring directly.
- Add a probability/confidence score alongside the binary verdict, and track prediction history for a given location over time.

11. APPENDIX

Source Code

Key project files (full source included in the submitted project archive):

- `data/generate_dataset.py` — synthetic dataset generation
- `model/train_model.py` — EDA, preprocessing, model comparison, and persistence
- `app/app.py` — Flask web application and inference routes
- `app/templates/` — `home.html`, `index.html`, `chance.html`, `no_chance.html`
- `app/static/` — `main.css`, `main.js`
- `requirements.txt` — pinned dependency versions (NumPy, Pandas, Scikit-learn, XGBoost, Matplotlib, Seaborn, Flask, Joblib)

Dataset Link

Reference dataset: Kaggle — "Rainfall Dataset" (kaggle.com/arbethi/rainfall-dataset). A statistically realistic synthetic equivalent with the same schema (AnnualRainfall, SeasonalRainfall, CloudVisibility, Temperature, Humidity → class) is included in this project as `data/flood_dataset.csv` and can be regenerated via `data/generate_dataset.py`.

GitHub & Project Demo Link

GitHub Repository:

Project Demo (video / hosted link):